Date of publication xxxx 00, 0000, date of current version 22.05.2026.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Large-scale workflow placement in serverless computing using integer nonlinear programming

JOSHUA ADAMEK[1], NATALIE CARL[2], TREVER SCHIRMER[2], MORITZ HEINLEIN[1] DAVID BERMBACH[2] and Sergio Lucia[1], (Member, IEEE)

[1]Chair of Process Automation Systems, TU Dortmund University, 44227 Dortmund, Germany (e-mail: joshua.adamek@tu-dortmund.de, moritz.heinlein@tu-dortmund.de, sergio.lucia@tu-dortmund.de)

[2]Scalable Software Systems Research Group, Technische Universität Berlin, 10623 Berlin, Germany (e-mail:carl@tu-berlin.de, t.schirmer@tu-berlin.de, db@3s.tu-berlin.de)

Corresponding author: Joshua Adamek (e-mail: joshua.adamek@tu-dortmund.de).

*This work has been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - Projektnummer 495343202.

ABSTRACT Serverless edge computing has become a powerful cloud framework that enables the execution of large workflows without the need for the user to manage the underlying servers and edge devices. In this work, we address the challenge of deploying these workflows on existing servers and edge devices such that monetary costs for the users and workflow evaluation times are minimized. To this end, the workflow and cloud node attributes are modeled in a mathematical framework. As a result, we model for the first time the optimal placement problem as a nonlinear integer program. To solve both the issues of scaling towards a larger number of cloud/edge nodes as well as decomposed knowledge of node attributes, we propose a decomposition strategy. In a case study, we show the advantages in computation time and costs of the proposed approach.

INDEX TERMS Cloud computing; Optimal Scheduling; Integer programming

I. INTRODUCTION

SERVERLESS computing has emerged as a promising cloud computing paradigm for Internet of Things (IoT) applications [15]. As the user interaction with the cloud platform is decoupled from the underlying physical servers, this offers flexibility and easier usage. This general idea motivates a Functions-as-a-Service (FaaS) architecture [24], where deployment requires only functions and their data or software dependencies, while placement on physical devices is managed entirely by the cloud management system [5]. FaaS platforms have gained attention in both research [3] and practice [17], particularly for IoT applications involving large numbers of dependent functions, also referred to as workflows [11, 10]. The decomposition of workflows into functions and their deployment represents one of the primary use cases for FaaS [21]. Previous methods focus on the online scheduling of user requests [23, 1]. The deployment of the workflow functions, that is placing the functions on the serverless architecture before they are invoked by the user, has only recently become of increased interest [16, 4]. In this work, we therefore distinguish between the placement problem during workflow deployment and the selection process during online operation [16]. Solving the placement problem requires knowledge of the selection strategy beforehand since overall cost depends strongly on how users access deployed workflows. Existing research addresses this through fixed strategies [23] or adaptive probabilistic strategies during online execution, as in [22]. The deployment has become increasingly complex as FaaS architectures evolve from centralized server structures toward distributed deployments across multiple locations and providers. Furthermore, edge and fog computing devices [2, 14] promise reduced latency

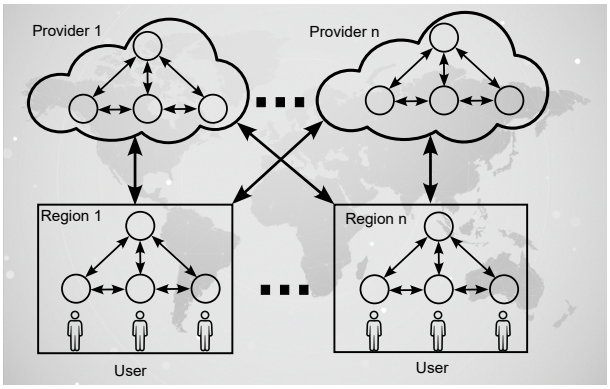


FIGURE 1: Serverless architecture considered in this work within a FaaS structure.

but introduce large numbers of potential compute nodes for function placement. While heuristic methods exist for this problem [17], the growing complexity calls for more rigorous optimization approaches.

Recent research focuses on frameworks that optimize placement decisions based on specific objectives such as execution time, user cost, or energy consumption of cloud nodes [16, 19]. These scheduling techniques typically involve integer decision variables, e.g., whether a function is deployed on a node or not. The computation time for integer optimization problems scales exponentially with problem size. Consequently, many scheduling frameworks rely on linear formulations [6] that can be handled efficiently using established methods [8]. Advances in nonlinear integer programming have been achieved by solvers such as Gurobi [7] and BARON [20], though these approaches still struggle with large-scale problems [9]. For this reason, existing approaches to placement and selection focus on learning-based solutions, such as reinforcement learning, which aims to find optimal policies through extensive simulation but suffer as well from exponential runtime growth and limited scalability to large numbers of nodes. For instance, the authors in [23] propose an optimal scheduling framework using reinforcement learning for serverless cloud/edge applications with a maximum of eight nodes.

This paper develops a mathematical framework to solve the workflow placement problem in large-scale serverless architectures. The contribution lies first in proposing a realistic nonlinear model that incorporates parallel execution branches within workflows, which have not been captured by previous models and is a pivotal step towards the description of real-world, larger workflows. In addition, we present a new decomposition method to improve scalability. The regional structure of cloud providers, as described in Figure 1, enables solving each regional placement problem independently. This approach also mitigates assumptions made by previous optimization-based methods that rely on centralized information across multi-provider networks. Finally, scalability and performance are evaluated through simulated case studies. The remainder of this paper is organized as follows: Section II introduces the modeling framework, including attributes for workflows and nodes, Section III formulates the optimal placement problem and presents a decomposed solution enabling scalable algorithms for distributed clouds. Section IV provides case studies demonstrating advantages such as scalability to larger problems, adaptability in multi-objective settings, and comparison to heuristic approaches. Section V concludes with final remarks and outlook.

II. MODELING OF PLACEMENT AND SELECTION OF WORKFLOWS

A. WORKFLOW MODELING

In the following, we establish the modeling framework for the workflows considered in this work. A workflow s consists of N_{func} functions. Each workflow is deployed to the cloud/edge structure $n_{\text{deployments}}$ times. Each function is stateless [5], meaning that all required data dependencies and inputs are provided externally, and a function is executed only once within a workflow. We assume that every function in a workflow is executed each time the workflow is invoked. The workflow structure is illustrated in Fig. 2. We allow for N_{branches} different branches within each workflow. To define a workflow, we introduce variables $b_{c,m}$, which equals one if function $m \in \{0, \dots N_{\text{func}} - 1\}$ belongs to branch $c \in \{0, N_{\text{branches}} - 1\}$. Furthermore, we set $a_{m,\tilde{m}} = 1$ if function $\tilde{m}$ follows function m, indicating that function m must be executed before data can be transferred between these two functions. Each function has an estimated computational demand $R_{\text{func},m}$, defined by its computation time $t_{\text{func},m}$ and memory usage RAM_m. This demand is assumed to be fixed such that

$$R_{\text{func},m} = \text{RAM}_m t_{\text{func},m} \,,$$

which is treated as a constant value.

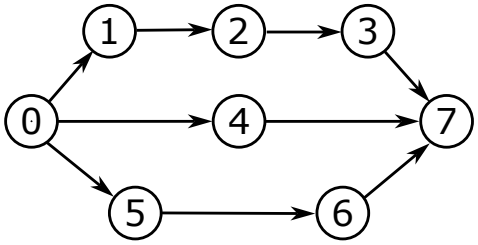
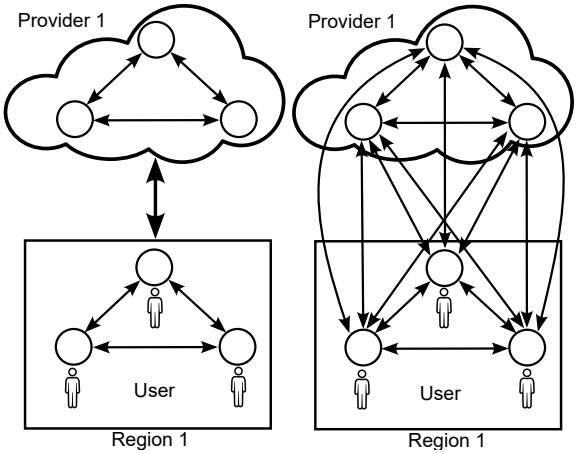


FIGURE 2: Structure of a workflow with eight functions and three branches. In this example, $b_{0,1} = 1$ indicates that function 1 belongs to branch 0, while $a_{1,2} = 1$ shows that function 2 succeeds function 1.

In addition to computational demand, each function sends data d_{sent} to its successor within the respective branch once execution finishes. The first function of a workflow receives input data d_{in} from the user invoking the workflow. Figure 2 illustrates these data flows required for completing the workflow. Furthermore, each function might require dependency data $d_{\text{dependencies}}$, which are stored physically on nodes within the serverless architecture.

B. SERVERLESS INFRASTRUCTURE MODELING

The task addressed by the placement problem is to assign each function m of every workflow n to a physical device within the serverless structure, referred to as a node. We define the serverless structure as a set of nodes $i \in \{0, \dots, N_{\text{nodes}}\}$. These nodes are divided into edge nodes and cloud nodes. In this modeling framework, cloud nodes provide higher computational power, which can reduce the computation time of a function m. In our approach, we assume that executing functions on cloud nodes incurs monetary cost, whereas computations on edge nodes are considered free, but the modeling approach is not limited to this assumption. Furthermore, we consider multiple participants in the cloud environment. Each cloud node belongs to an entity (e.g. company or provider) that defines its attributes. For each provider, we assume a



(a) Assumed model of connections in the serverless architecture.

(b) Non-scalable model of connections in the serverless architecture.

FIGURE 3: General connection structure of the serverless architecture. Cloud providers and edge nodes are distinguished. Requests from users originate from edge nodes only. Cloud nodes are clustered by provider or region. Connections between providers or regions exist through single links, as shown on the left. A fully connected topology, as shown on the right, is neither efficient nor realistic when different providers are involved.

constant price to execute one megabyte for one second on its devices, denoted by p_{RAM}. This assumes a linear pricing model for the number of CPUs involved in executing a function. Additionally, we define a constant price for sending or receiving one megabyte of data as p_{send}, and a cost for transferring dependency data required for function execution as $p_{\text{dependencies}}$ per megabyte.

Figure 3 illustrates the proposed structure and connectivity of the serverless architecture using a simplified example. The bottom level represents user nodes at the edge from which requests originate during workflow execution. Each connection between node i and node j is characterized by an estimated latency $L_{i,j}$. As shown in Fig. 3b, an idealized setting would allow all nodes to be fully connected so that data could be transferred freely among them. In practice, especially with many providers and users, a separation occurs as depicted in Fig. 3a. It is reasonable to assume that regions or providers can communicate only through single head node links connecting their respective top-level representatives. This leads naturally to a hierarchical decomposition into regions and providers. In the simple example, there are two levels in the hierarchy: level zero contains two head nodes (region one and provider one), while level one includes three subordinate nodes per region/provider. We now establish notation for indexing these hierarchical levels in general form with levels indexed by $l \in \{0, \dots, n_l - 1\}$. At each level l, we consider $n_{\text{regions},l}$ regions/providers indexed by $r \in \{0, \dots, n_{\text{regions},l} - 1\}$, each containing $n_{\text{nodes},l,r}$ individual nodes indexed by

Algorithm 1 Latency calculation for regionally structured serverless architectures.

Require: Indices i and j of the physical devices
  $L_{i,j} = 0$ and $l = n_l - 1$
  $\tilde{i} = \text{curr_node}(l, i)$, and $\tilde{j} = \text{curr_node}(l, j)$
  $s_i = \text{curr_region}(l, i)$ and $s_j = \text{curr_region}(l, j)$
  while $s_i \neq s_j$ **do**
    $L_{i,j} = L_{i,j} + L_{l,r_i,0,\tilde{i}} + L_{l,r_j,0,\tilde{j}}$
    $\tilde{i} = \text{succ_node}(l, r_i)$ and $\tilde{j} = \text{succ_node}(l, r_j)$
    $r_i = \text{succ_region}(l, r_i)$ and $r_j = \text{succ_region}(l, r_j)$
  $L_{i,j} = L_{i,j} + L_{l,r_i,\tilde{i},\tilde{j}}$

$i \in \{0, \dots, n_{\text{nodes},l,r} - 1\}$. The number of regions at level l is therefore the number of nodes at level $(l-1)$:

$$n_{\text{regions},l} = \sum_{s=0}^{n_{\text{regions},l-1}} n_{\text{nodes},l-1,r} \,.$$

In this setting, latencies are defined for every connection between two nodes within the same region such that $L_{l,r,i,j}$ denotes the latency between nodes i and j in region s at level r. To navigate within the regional structure we define the following four functions: Given a node i in level l (that is the overall node number in the entire level over all regions in this level), we define the function $r = \text{curr_region}(l, i)$ that outputs the current region of this node, while $\tilde{i} = \text{curr_node}(l, i)$ is the index of the node within this region (that is the local node number in this region starting from the index zero in each region). Likewise, for a region s in level r, we define the successor node $\tilde{i} = \text{succ_node}(l, r)$ in level $l - 1$, which is the index of the node in the successor region $\tilde{r} = \text{succ_region}(l, r)$. The latencies between two nodes i and j can be calculated according to Algorithm 1.

III. SOLUTION STRATEGY

A. DECISION VARIABLES AND ASSUMED PROBLEM STRUCTURE

To set up the placement problem, the optimization variables defining the placement must be established. Since each workflow t is deployed separately, we omit the index t in our problem formulation for brevity. Each function m of workflow n must be placed on a node i within the serverless architecture. Therefore, we define the decision variable $P_{n,m,i}$ to be one if this function is placed on node i, and zero otherwise. For every function, exactly one node is selected, such that

$$\sum_i P_{n,m,i} = 1 \,,$$

for all functions m in workflow n. The main difficulty in describing the cost function arises from the fact that total cost does not directly result from workflow placement itself but rather from user selection of workflows during execution. Consequently, we aim to minimize the expected selection

cost incurred by users invoking workflows. To model user behavior, we define a variable $H_{k,n}$ indicating which workflow is selected by user k. In general, this variable is not known in advance. It represents a runtime decision made during operation. Optimal choices may differ depending on request conditions. Modeling such behavior leads to a two-stage stochastic optimization problem [18] , where placement variables $P_{n,m,i}$ would need to be optimized for all possible realizations of $H_{k,n}$. Since integer-based two-stage stochastic problems are computationally demanding [18] , they are unsuitable for large-scale serverless architectures. We therefore assume a deterministic selection policy $H_{k,n}$, which we determine prior to execution and include as decision variables in the placement problem, ensuring that exactly one workflow is selected for each user request:

$$\sum_{n} H_{k,n} = 1 \,,$$

for every possible user k. In turn, this policy cannot react to the stochastic nature of the user requests, leading to possible blocking of resources or deployed functions, amplifying the need to address this issue in the placement problem as proposed in the next subsection.

B. PROBLEM DEFINITION

After modeling the workflows and nodes in the serverless architecture, we next define costs and constraints for the placement problem. We place each workflow separately, that is, each workflow t is placed by solving the following optimization problem. We consider two competing objectives that influence the placement: the time required to evaluate a workflow and the monetary cost incurred by a user when executing it.

Using the modeling introduced in the previous section, these costs can be expressed as functions of the placement and selection decision variables, as well as parameters describing nodes and workflows (summarized in Table 1). We denote by $T_{n,c}$ the time required to evaluate branch c of workflow n. The overall evaluation time depends on the slowest branch among all $n_{\text{branches},s}$ branches of a workflow. To include this dependency in the optimization problem, we introduce an auxiliary variable T_n, constrained by

$$T_n \geq T_{n,c} \quad \forall c \in \{0, N_{\text{branches},s}\} \,.$$

Each branch time $T_{n,c}$ is computed as the sum of function evaluation times for all functions m within that branch and latencies between nodes hosting consecutive functions. Consequently, we can formulate

$$T_{n,c} = \sum_{m}\sum_{i}(b_{c,m}P_{n,m,i}t_{m,i} + \sum_{\tilde{m}}\sum_{j}P_{n,m,i}P_{n,\tilde{m},j}a_{m,\tilde{m}}b_{c,m}L_{i,j}) \,.$$

Note that latency introduces bilinear terms into this expression, as the placement variables $P_{n,m,i}$ and $P_{n,\tilde{m},j}$ are multiplied. Analogously, monetary costs consist of two components: a linear part representing direct execution costs $C_{m,i}$ for running function m on node i, and a bilinear part describing data transfer costs between adjacent functions deployed on different nodes. We denote $C_{m,i}$ the sum of the execution cost and the potential additional cost for transferring data to node i

$$C_{m,i} = p_{\text{RAM},i}R_{\text{func},m} + p_{\text{data},i}d_{\text{data},m} \,.$$

If the transfer data is already available on node i, we formulate $p_{\text{data},i} = 0$. Additionally, data is transferred between two functions located on different nodes, represented by

$$D_{m,\tilde{m},i,j} = (p_{\text{send},i} + p_{\text{send},j})d_{\text{sent},m} \,.$$

In addition to these two primary cost components, time and monetary, we must also account for communication costs between users and nodes executing either the initial or final function of each workflow. These contributions are added respectively to both time-related and monetary terms. Furthermore, all these costs results from invoking these workflows from user requests. We assume that the user randomly requests workflows with a rate of λ_k for every leaf node k. To get the correct expected cost, we sum over user requests

TABLE 1: Overview of the model parameters used in the main optimization problem (1).

Notation	Definition
k	Index used to address leaves
i,j	Indices used to address general nodes in the cloud
s	Index used to address different workflows
l	Index used to address levels in the serverless architecture
m	Index used to address functions within a workflow
n	Index used to address different deployments of the same workflow
r	Index used to address a region in a level in the serverless architecture
c	Index used to address the branches of the workflow
$H_{k,n}$	Variable selecting workflow n for a request from leaf k
$P_{n,m,i}$	Variable placing function m from workflow n on the node i
$C_{m,i}$	Cost to execute function m on node i
$D_{m,i,j}$	Cost to send data generated by function m from node i to j
T_n	Maximum time to execute workflow n
$T_{m,i}$	Time to execute function m on node i
$L_{r,s.i,j}$	Latency between node i and j in region s in level r of the serverless architecture
p_{RAM}	Cost to execute one MB of RAM for one second
p_{send}	Cost to send 1 MB of output data from a function
p_{data}	Cost to transfer 1 MB of data needed for the execution of a function
d_{data}	Amount of data needed for the execution of a function
d_{send}	Amount of output data send from a function
$t_{\text{func},m}$	Estimated running time of function m on a normal device
RAM_m	Estimated RAM usage of function m
$R_{\text{func},m}$	Constant computational demand of function m
n_{func}	Number of functions in a workflow
$n_{\text{branches},s}$	Number of branches of workflow s
$n_{\text{deployments}}$	Number of deployments of workflow s
$b_{c,m}$	Indicates if function m is in branch c
$a_{m,\tilde{m}}$	Indicates if function $\tilde{m}$ is a successor of function m
$x_{i,m}$	Estimated speedup of large functions deployed on cloud node m
n_l	Number of levels considered in the serverless architecture
$n_{\text{regions},l}$	Number of regions considered in level l in the serverless architecture
$n_{\text{nodes},l,r}$	Number of nodes in region r on level l in the serverless architecture

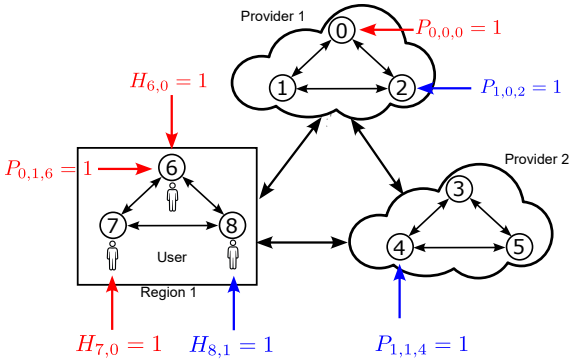


FIGURE 4: Overview of the centralized optimization strategy on the example of a workflow consisting of two functions placed two times on an architecture with three edge nodes and two providers with three physical devices. The placement variables P show which function of which workflow is placed on which device, while the selection variable H defines which workflow will be selected from a user node. The first two users (node 6 and 7) select the first workflow (variables in red) while the third user (node 8) select the second workflow (variables in blue).

by weighting each monetary cost and evaluation time by the request rate λ_k for every user.

These two cost factors are exact if every resource and function is always free to use, that is, we do not expect waiting time in the scheduling approach due to resource constraints. As this is not realistic, we introduce an additional cost term addressing resource limitations. Because exact sequences of user requests cannot be anticipated during runtime, imposing strict constraints on workflow or node usage would lead to conservative solutions. Instead, we define a third penalty term that discourages excessive utilization of specific workflows or nodes within the serverless architecture. Additionally, modeling the exact waiting time anticipated due to the utilization of nodes and functions is equally unrealistic. Instead, we penalize the unequal placement of functions upon the serverless architecture, especially edge nodes that are resource constraint by a maximum RAM $\text{RAM}_{\text{max},i}$. We penalize the expected utilization of each node M_i where

$$M_i = \frac{\sum_k \sum_n \sum_m \lambda_k H_{k,n} P_{n,m,i} R_m}{\text{RAM}_{\text{max},i}} + M_{0,i}\,.$$

The utilization of this resource is dependent on the usage of this node over all s workflows. To mitigate this, we track the utilization by adding the current utilization to $M_{i,0}$, which in turn is the summed utilization of previous placed workflows. Furthermore, blocking of a function happens if a specific function is already used through a different request. We therefore penalize the function utilization given as

$$p_{n,m} = \sum_k \sum_i \lambda_k H_{k,n} P_{n,m,i} t_{m,i}\,.$$

As illustrated in Figure 4, the total cost is evaluated as the summation over all potential user requests.The three individual cost components, monetary, temporal, and utilization-related, together form a multi-objective optimization problem. We solve this problem by employing a weighted-sum approach [12], assigning each objective a corresponding weight. Consequently, the complete optimization problem can be formulated as in (1).

Every cost is nonlinear, due to the multiplication of the selection and placement optimization variables. Additionally, nonlinear terms are added in the constraints as we model time and monetary costs based on the interaction of adjacent functions on possible different nodes. We further introduce continuous variables such as the monetary bilinear term S and the basic time variable T. The problem we are solving is therefore classified as a mixed-integer nonlinear integer problem. This problem formulation scales exponentially in computation time for an increasing number of nodes and workflow functions, which is especially difficult for a serverless architecture with many nodes. Therefore, and considering the already established decompositions that are build into the modeling of the serverless architecture, we propose a decomposition approach that scales well for a larger number of nodes in the next subsection.

C. DECOMPOSITION STRATEGY

There are two main reasons why solving (1) is computationally intractable in practice. First, it requires complete knowledge of all costs, speedups, and latencies across the entire serverless architecture on a single node. This assumption is unrealistic since nodes may belong to different providers that do not share all information, and collecting such statistics would accumulate large amounts of data on one node. Second, computation time scales exponentially with the number of nodes considered. As described in Subsection II-B, direct communication among all nodes as a fully connected network is improbable. Instead, it is more realistic to divide the architecture into smaller regions represented on the higher level as a single node. We extend this idea to the optimization problem itself. On each hierarchical level, placement decisions for functions within a region are performed by the respective head node that possesses local information such as latencies, costs, and speedups for its region. These regions are chosen such that node characteristics (speedup factors, type of node, region) remain homogeneous within each subset, and edge nodes are grouped only with other edge nodes. This approach addresses both challenges: only local information is required, and regions remain small enough for efficient computation. An example of the structure of the decomposition is illustrated in Figure 5. The goal of the decomposition approach is to solve a variant of (1) independently within each region at every level while maintaining proximity to the solution obtained when considering all nodes simultaneously. Therefore, we now reformulate the decision variables to $H_{l,r,k,n}$ and $P_{l,r,m,n,i}$ as we consider $r \in \{0,\ldots,n_{\text{regions},l} - 1\}$ optimization problems in level $l \in \{0,\ldots,n_l - 1\}$. Figure 5 illustrates the decomposition strategy. We start at level $n_l - 1$, where physical nodes exist. For the next level, groups of edge-node

$$\min_{P,T,H,S} \underbrace{w_1 \sum_k \sum_n \lambda_k H_{k,n} \left(P_{n,m_{\text{start}},i} D_{\text{in},i} + P_{n,m_{\text{end}},i} D_{m,i,k} + \sum_i \sum_m P_{n,m,i} C_{m,i} + S_{n,m,i} \right)}_{\text{Monetary costs } M}$$

$$+ \underbrace{w_2 \sum_k \sum_n \lambda_k H_{k,n} \left(T_n + \sum_i \left(P_{n,m_{\text{start}},i} L_{k,i} + P_{n,m_{\text{end}},i} L_{k,i} \right) \right)}_{\text{Basic Time cost } B} + \underbrace{w_3 \left(\sum_n \sum_m (u_{n,m})^2 + \sum_i M_i^2 \right)}_{\text{Utilization cost } U}$$

subject to:

$$\underbrace{\sum_i P_{n,m,i} = 1 \ \forall (n,m) \quad \sum_n H_{k,n} = 1 \ \forall (k)}_{\text{Placement and selection constraint}} \quad \underbrace{S_{n,m,i} = \sum_{\tilde{m}} \sum_j P_{n,m,i} P_{n,\tilde{m},j} a_{m,\tilde{m}} D_{m,i,j} \ \forall (n,m,i)}_{\text{Monetary bilinear variable}} \tag{1}$$

$$\underbrace{T_n \geq \sum_m \sum_i (b_{c,m} P_{n,m,i} t_{m,i} + \sum_{\tilde{m}} \sum_j P_{n,m,i} P_{n,\tilde{m},j} a_{m,\tilde{m}} b_{c,m} L_{i,j}) \ \forall (n,c)}_{\text{Maximum branch time constraint}}$$

$$\underbrace{M_i = \frac{\sum_k \sum_n \sum_m \lambda_k H_{k,n} P_{n,m,i} R_m}{\text{RAM}_{\max,i}} + M_{0,i} \ \forall (i)}_{\text{Node utilization}} \quad \underbrace{u_{n,m} = \sum_k \lambda_k H_{k,n} P_{n,m,i} t_{m,i} \ \forall (i)}_{\text{Function utilization}}$$

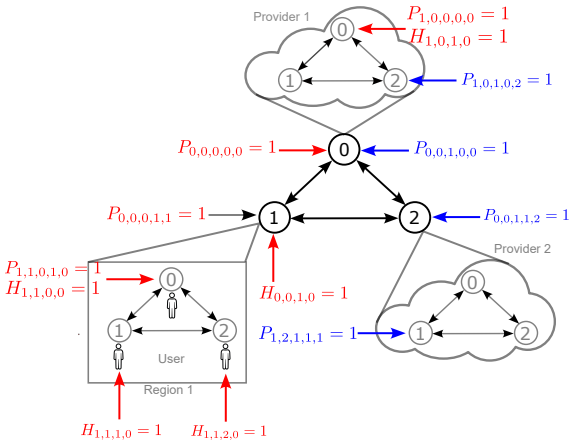


FIGURE 5: Decomposed approximation based on regions and providers shown for three nodes per provider and three edge-region nodes. The one workflow in this example is placed twice $n_{\text{deployments}=2}$. Each region or provider is represented by a single node in the main problem, where placement variables P and selection variables H are determined. Functions placed on representing nodes are then resolved within subproblems for their respective subsets. In contrast to the centralized approach, selection can be performed only once per region. Thus, only one workflow can be selected for all three edge nodes collectively in this example.

regions and cloud providers are aggregated into single representative nodes. This results in several regions, each having a head node responsible for solving its local subproblem. Analogous to the main problem formulation, user requests must be included for every subproblem. Any node consisting solely of edge devices or representing physical edge hardware remains treated as a leaf node from which requests originate. Therefore, the user request rate is calculated by summing over all rates in the lower level

$$\lambda_{l-1,\text{succ_node}(k)} = \sum_k \lambda_{l,k} \,. \tag{2}$$

Each region of nodes at level l corresponds to a single representative node at level $(l-1)$. Therefore, placements within regions can be tracked by determining whether function m of workflow n was placed on its predecessor node at level $(l-1)$. Naturally, we start solving the placement problem at level 0 and proceed through successive levels by providing the upper level placements to the lower level head nodes. Although this introduces an exponential increase in subproblem count across levels, each subproblem operates independently on its own head node and can thus be computed in parallel. Two additional constraints must be introduced to correctly perform decomposition: First, if certain functions or entire workflows have already been placed elsewhere, that is, if these functions were not assigned to their predecessor node at level $(l-1)$, they cannot be reconsidered within the current subproblems. Because communication routes always pass through head nodes, placing such functions virtually on their head node has an equivalent effect as placing them outside their subset entirely (see Figure 5). Therefore, we add a function m into the set $\mathcal{M}$ if $P_{l-1,\text{succ_region}(r),n,m,\text{succ_node}(r)}$=0 and add the following constraints

$$P_{l,r,n,m,0} = 1 \ \forall m \in \mathcal{M} \,. \tag{3}$$

As these are virtual placements, they cannot be included in the calculation of the selection cost, especially the function placement penalization $M_{l,r,0}$ on the head node. To account

for this number of virtual placements in the selection cost, we have to adapt the function term $M_{l,r,0}$ for all subproblems

$$M_{l,r,0} = \frac{\sum_k \sum_n \sum_m \lambda_{l,r,k} H_{l,r,k,n} P_{l,r,n,m,0} R_m}{\mathrm{RAM}_{\max,i}} - \frac{\sum_k \sum_n \sum_{m^\star} \lambda_{l,r,k} H_{l,r,k,n} P_{l,r,n,m^\star,0} R^\star_m}{\mathrm{RAM}_{\max,i}} + M_{l,r,0,0} \,, \tag{4}$$

where $m^\star \in \mathcal{M}$. If complete workflows have been deployed elsewhere, we reduce $n_{\text{deployments}}$ by one accordingly. This pruning strategy reduces computational effort since regions without placed functions do not need to perform further calculations. Under current modeling assumptions, we expect no cost increase due to this part of the decomposition. The situation differs for selection decision variables $H_{l,r,k,n}$. Because upper-level representations also aggregate multiple edge regions into regional entities, selection decisions become coupled among users belonging to those regions (see Figure 5). To obtain correct cost estimation during selection evaluation, we must assume identical workflow choices across all user requests originating from one region. Hence, recomputation of $H_{l,r,k,n}$ beyond level $l > 0$ is omitted. Again, we have to distinguish between leaf nodes that stem from real users, where if in the upper level succ_node(r) is a leaf node and $H_{l-1,\text{succ_region}(r),\text{succ_node}(r),n} = 1$ we set

$$H_{l,r,k,n} = 1 \,, \tag{5}$$

for all leaf nodes k participating in the next-level subproblems. For the head node zero, we have to additionally account for workflows that are possibly invoked through nodes from other regions. As these information exchanges run through this main node, we treat the main node as the leaf for these additional nodes. Therefore, we sum the selections $H_{l-1,\text{succ_region}(r),k,n} = 1$ for all leaves that are not the successor of this optimization problem that is

$$H_{l,r,0,n} = \Gamma\left(\sum_k H_{l-1,\text{succ_region}(r),k,n}\right) \,, \tag{6}$$

where $\Gamma(\cdot)$ is the step function. This coupling reduces degrees of freedom compared with centralized optimization and typically increases overall cost, particularly regarding selection-related terms, since regional uniformity restricts flexibility among users sharing similar locations or providers. Therefore, successful decomposition requires balancing computational feasibility against model accuracy, specifically, there exists a trade-off between reduced computational demand achieved via coarse regional aggregation (solving fewer large-scale problems) and finer granularity necessary to represent realistic distributions among edge-node clusters. During the solving process, the user requests have to be adapted as well, depending on the decisions made in the upper level. As the head node serves as a placeholder for requests coming from outside the region, we have to additionally add the user request rate depending on the requests from the other regions, that is

$$\lambda_{l,0} = \lambda_{l,0} + \sum_k \lambda_{l-1,k} - \lambda_{l-1,\text{succ_node}(0)} \,, \tag{7}$$

where we sum over all other regions in the upper level to gather the requests. With the addition of (2)-(7), the main problem (1) can now be solved for each region r in each level l, requiring only communication down the levels during the placement and an update of the current cloud attributes after the placement on the lowest physical node level.

IV. CASE STUDY

A. MODELING PARAMETERS AND SIMULATION

In the case study, we analyze the benefits of the proposed algorithm using simulated serverless architectures and synthetic workflows. For modeling purposes, we define parameters describing both workflows and the architecture, such as evaluation costs on cloud nodes, the number of nodes within the serverless architecture, and the number of functions contained in each workflow. These parameters are either uniformly distributed within fixed bounds or remain constant throughout the case study. Table 2 defines these parameters together with the values used across all experiments. The serverless architecture is modeled with two cloud providers and one type of edge node. Each provider has fixed costs for sending data, executing functions, and transferring dependent data, chosen to approximate realistic values for commercial cloud services [23]. Additional architectural parameters include bounds for latencies and speedups on nodes. To achieve a more realistic simulation, latency bounds are reduced in each level to reflect increasingly regional connections according to the following formula

$$L_{l,\text{upper bound}} = L_{\text{nodes}} b^{-l}_{\text{decrease}} \,.$$

Function attributes are randomly sampled to generate diverse workflows. These attributes include evaluation time requirements, data sizes exchanged between functions, dependency data sizes, and the probability that a function depends on external data. In addition to these workflow parameters, we define several architectural scaling parameters that influence problem size. The number of nodes per level is uniformly distributed, combined with the total number of levels, these

TABLE 2: Fixed model parameters for the serverless architecture and workflows.

Architecture	Value	Workflow	Value
p_{RAM} provider 1	$0.0167\,\$\text{MB}^{-1}$	$n_{\text{deployments}}$	[2 5]
p_{send} provider 1	$0.01\,\$\text{MB}^{-1}$	RAM	$[1\ 100]\text{MBs}^{-1}$
p_{data} provider 1	$0.01\,\$\text{MB}^{-1}$	t_{func}	[100, 1000]s
p_{RAM} provider 2	$0.02\,\$\text{MB}^{-1}$	x_{func}	[0.75, 1]
p_{send} provider 2	$5 \cdot 10^{-3}\$\text{MB}^{-1}$	d_{input}	[0, 100]MB
p_{data} provider 2	$1 \cdot 10^{-3}\$\text{MB}^{-1}$	d_{send}	[100, 1000]MB
L_{nodes}	[0, 10]s	d_{data}	[0, 1000]MB
b_{decrease}	10	ρ_{data}	0.2
n_l	2	n_{func}	[3, 6]
λ_k	[0.03 0.05]		

two parameters determine the overall scale of the serverless architecture. Furthermore, we separately specify the number of nodes considered in the highest level. For workflows, scalability is controlled by varying both the total number of workflows deployed on the architecture and the number of functions contained within each workflow. As the actual costs for a placed workflow can only be evaluated in the online selection phase, the case study simulates requests in a simulation time of $t_{\text{sim}} = 200\,\text{s}$. The requests are randomly sampled with a Poisson distribution using the request rate λ_k for each physical user k. These requests are scheduled using a FIFO scheduling, where for each workflow request, the future resources are blocked immediately. In this simulation, we therefore model two blocking mechanisms: A specific function m in a workflow t with the deployment number n cannot be used twice in the same time step, and if the RAM resource of node i is fully occupied, no further function can be executed on said node. Instead of the heuristic utilization cost U that is used in the placement problem, we focus, therefore, on the actual waiting time W due to blocking. We can therefore formulate the overall cost C, defined as a weighted sum of the two competing objectives,

$$C = w_1 M + w_2 (B + W)\,.$$

In this study, we additionally will have to show that the weight w_3 has an influence on reducing the waiting time. For comparison reasons, we frequently evaluate the gap between the cost of two different solution strategies

$$g = \frac{C_1 - C_2}{C_1}\,,$$

with $C_1 > C_2$. Second, we analyze the computation time required to solve the placement problem. In the centralized formulation, this corresponds to solving a single global optimization problem. In contrast, under decomposition we assume that total computation time equals the sum over maximum subproblem times at each level,

$$t_c = \sum_{l=0}^{N_{\text{level}}-1} \max_r t_{c,r,l}\,,$$

where $t_{c,r,l}$ denotes computation time for subproblem r at level l. Since subproblems within a given level can be solved in parallel, only their maximum runtime contributes per level. The communication time required to pass information between levels is neglected here, as the centralized approach would also need constant communication between the nodes to update the central parameters and because we want to focus on the computational burden of the placement optimization problem in itself. All simulations are performed on an AMD Ryzen 9 16-Core processor equipped with 94 GB RAM. The modeling framework and cost calculations are implemented in Python. Integer decision problems are solved using Gurobi [7]. The code necessary to generate the presented results is openly available[1].

[1] https://github.com/JoshuaAda/IEA_LSWPISCUINP

B. SMALL WORKING EXAMPLE

First, we reintroduce the small example used in the modeling section, now with real parameters as described in Table 2 to illustrate the modeling and solution structure of the proposed placement algorithm and highlight its differences compared to the centralized approach. For ease of visualization, we reduce the number of deployments to two, the number of functions for each workflow to three, the number of nodes in the second level to three, and the number of edge node regions in the first level to one. Figure 6 shows the full problem, considering all nodes in one optimization problem. In comparison, Figure 7 shows the serverless architecture considered in the decomposed setting, consisting of two hierarchical levels.

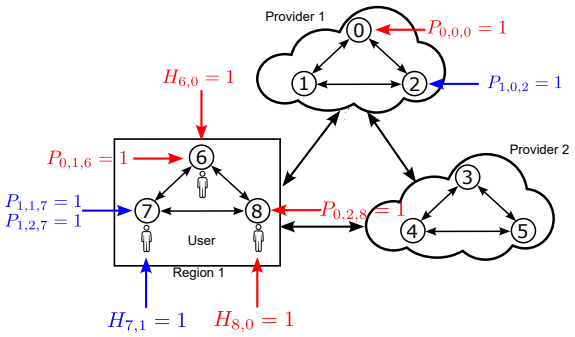


FIGURE 6: Placement solution of the centralized approach for two workflows (first workflow placements and selections in red, second in blue) containing three functions. In the centralized approach, the three different users can select different workflows (user 6 and 8 selecting the first, user 7 the second), giving more freedom to reduce blocking as well as basic time and monetary costs.

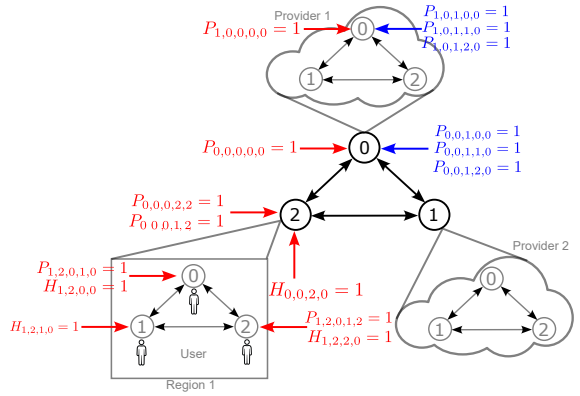


FIGURE 7: Placement solution of the decomposed approach for two workflows (first workflow placements and selections in red, second in blue) containing three functions. In the decomposed approach, only one workflow can be selected as the three users are lumped in one region on the first level of optimization. The functions are placed on the regions in the first level and are later distributed onto the nodes of the regions in the second level.

In the centralized case, three users can send requests independently, whereas in the decomposed approach, only one workflow can be selected per region, resulting in slightly higher overall cost. In total, we solve one optimization problem containing 131 decision variables for the centralized formulation and four subproblems for the decomposed approach. The main problem in the decomposition approach includes 49 decision variables, while the sizes of subordinate problems depend on decisions made at this top level (26, 26, and 53 decision variables for the specific subproblems). Figure 6 presents results for the centralized solution, while Figure 7 shows outcomes for the decomposed strategy assuming equal weights for the monetary cost and time cost $w_1 = w_2 = 0.5$, and a utilization cost of $w_3 = 5$. The primary difference between the centralized and the decomposed approach lies in the selection behavior. This difference can be seen in the small example. In Figure 7, all three users can select the first workflow, whereas the second workflow has no influence on the cost. On the contrary, the centralized approach can select different workflows for different users, such that the second workflow is selected for the second user. The result follows directly from the costs presented in (1). In general, it is beneficial to place the functions on the regional edge nodes as they offer small latencies and free evaluation. As the first function in the workflow can be completed much faster on the cloud node of the first provider, it is placed there. As the latencies between the regional nodes are very small, the selection cost leads to a more equal distribution of the function onto the edge nodes. The total cost C across all three users in the simulation amounts to 345.24 for the centralized approach and 403.74 for the decomposed variant. The main difference stems from the waiting time, which is 187 s in the decomposed case against 79 s in the centralized approach, which is due to the higher degree of freedom to choose the workflows in the centralized approach. Despite the higher cost due to restricted selection flexibility, computation time decreases even within this small-scale scenario substantially from 225.33 ms for centralized optimization to 50.11 ms using the decomposition.

C. SCALING OF THE DECOMPOSITION APPROACH

After demonstrating the main functionalities of the proposed algorithm, we now highlight its principal advantage: the significantly improved scalability of the algorithm compared to the centralized approach. This aspect is crucial for realistic workflow placement scenarios, as the number of nodes expected in practical serverless architectures is typically significantly larger than those considered in the previous subsection. To scale the architecture, two parameters can be adjusted. First, the number of nodes within each subregion can be increased either as a fixed value or as bounds for a uniform distribution from which these values are sampled, which leads to a moderate increase in total node count. Second, we can increase the number of hierarchical levels used to organize nodes within the architecture. This results in an exponential growth of total nodes and provides a useful setting for evaluating scalability under large system sizes. Due to this exponential growth in decision variables with increasing numbers of nodes, we expect that computation time for the centralized approach will scale poorly. Therefore, we compare both approaches using an architecture with two levels while gradually increasing node counts within subregions. Other workflow and node parameters are sampled uniformly within bounds provided in Table 2. Table 3 shows overall costs and computation times for both approaches. Costs are slightly higher for the decomposed method across all configurations, the maximum gap observed between both approaches is 6.17 %. For the centralized optimization, the runtime increases exponentially with the number of nodes. For instance, placing two workflows on a serverless architecture containing a total of 16 nodes already requires 590.29 s. In contrast, computation time for the decomposed approach increases logarithmically with node count (linear with respect to the number of levels) due to parallel evaluation of subproblems at each level.

TABLE 3: Overall cost C and computation time t_c for a two-level decomposition with an increasing number of nodes $n_{\text{nodes},1,r}$ in the second level for every region in comparison to the centralized approach that solves the full optimization problem.

Value	$n_{\text{nodes},1} = 2$	$n_{\text{nodes},1} = 3$	$n_{\text{nodes},1} = 4$
C [-] (decomposed)	16595.04	13618.46	15177.33
C [-] (centralized)	**15566.71**	**13515.05**	**14927.83**
g[%]	6.20	0.76	1.64
t_c (decomposed) [s]	**1.55**	**1.77**	**1.96**
t_c (centralized) [s]	25.63	280.97	590.29

These findings confirm that decomposition enables efficient scaling toward large architectures while providing accurate approximations to global optimal solutions. There is no clear increase or decrease of the gap between the centralized and decomposed solution with the rising number of nodes, confirming that the approximation error is not dependent from the number of nodes. The gap instead depends highly on the chosen scenario. For example, a rising number of deployments can allow for a much bigger gap due to the additional degree of freedom to choose workflows for the centralized approach as discussed in Subsection IV-B. For the computation time, we achieve a favorable scaling using the decomposed approach in contrast to the exponential growth in the centralized approach.

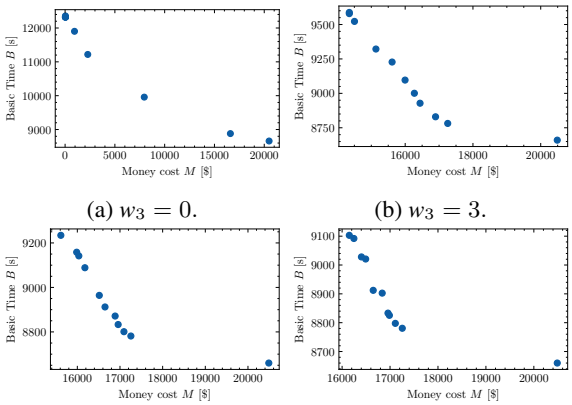
To further validate scalability benefits, we next increase architectural depth by varying hierarchical levels, thereby enabling substantially larger physical number of nodes. In this scenario, direct comparison against centralized optimization becomes infeasible since it fails to produce solutions. Instead, we benchmark against a simple heuristic strategy that places all functions on one cloud provider (provider 1), which serves as a standard baseline used, e.g., in [23]. In our case, it serves as a good heuristic, as blocking because of RAM constraints is reduced as much as possible with this heuristic. In this experiment, we scale from two up to four levels while as-

suming two regional edge nodes (and again two cloud nodes) initially. Each subproblem contains ten physical nodes per region or provider, consequently total node counts reach 40, 400, and 4000 physical devices at successive levels across three test cases. Table 4 shows the cost and computation times

TABLE 4: Overall cost C and computation time t_c for increased number of levels n_l and therefore the total number of nodes in the last level $n_\mathrm{nodes} = \sum_r n_{\mathrm{nodes},n_l-1,r}$ in the decomposition in comparison to a cloud only heuristic.

Value	$n_\mathrm{l}=2$ $N_\mathrm{nodes}=40$	$n_\mathrm{l}=3$ $N_\mathrm{nodes}=400$	$n_\mathrm{l}=4$ $N_\mathrm{nodes}=4000$
C [-] (decomposed)	**13668.11**	**11110.95**	**11553.51**
C [-] (heuristic)	16942.92	12141.04	13863.41
g[%]	19.33	8.48	16.66
t_c (decomposed) [s]	4.76	11.32	36.43
t_c (heuristic) [s]	**0.00**	**0.00**	**0.00**

for the decomposed approach and a cloud-only heuristic for an increased number of levels. It shows that the proposed decomposed approach consistently outperforms this heuristic regarding overall cost. Computation times reveal negligible runtime requirements for heuristics but linear growth for decomposition. This is an expected outcome since additional levels introduce more parallelizable subproblems rather than sequential dependencies. Overall findings demonstrate that employing optimal placement through integer programming becomes feasible even at large scales when combined with practical decomposition strategies and has significant advantages over simple heuristics rules. The proposed method thus compromised between theoretical optimality and computational tractability required for realistic serverless environments.



(a) $w_3 = 0$. (b) $w_3 = 3$.

(c) $w_3 = 6$. (d) $w_3 = 9$.

FIGURE 8: Pareto fronts for monetary and basic time costs using a weighted sum approach (increase weight w_1 for basic time cost, while decrease weight $w_2 = 1 - w_1$ for the monetary cost), exemplary for different values of penalization of utilization w_3.

D. SENSITIVITY TO WEIGHT PARAMETERS

Until this subsection, we have evaluated the solution of the placement problem with fixed weights. As described in (1), we consider two competing costs (monetary and time) with the additional utilization cost that in turn reduces the waiting time and therefore again the overall time to run a workflow. Next, we want to verify that monetary and time costs are competing and that the algorithm can be tuned using w_1 and w_2 to reduce or increase the monetary and basic time costs. Furthermore, we investigate whether increasing the utilization cost factor w_3 leads to a reduced waiting time and whether it has any impact on the tuning of the basic time and monetary cost. Understanding how varying the relative importance of each cost factor influences overall performance is essential when deciding which objective should receive greater emphasis. Such analyses are commonly conducted through Pareto front evaluation [13]. The Pareto front represents all points in the multi-dimensional cost space for which improvement in one objective necessarily leads to deterioration in at least one other. As proposed in [12], the optimal solution obtained from a weighted-sum formulation, such as the placement problem defined in (1), corresponds to a point located on this Pareto front. To further investigate trade-offs among monetary cost and evaluation time of the workflow, we perform a simulation involving ten workflows deployed on a serverless architecture with attributes given in Table 2. This experiment is conducted for different weights between the two competing costs (eleven values between zero and one for w_1 and $w_2 = 1 - w_1$). Additionally, this process is repeated for eleven additional values of w_3 between zero and ten.

TABLE 5: Overall waiting time in seconds evaluated with increased penalization of utilization w_3 for different weight combinations w_1/w_2 on the Pareto front.

	$w_3=0$	$w_3=3$	$w_3=6$	$w_3=9$
$w_1=0.0; w_2=1.0$	2175	1568	1568	1568
$w_1=0.3; w_2=0.7$	284136	1728	1576	1572
$w_1=0.6; w_2=0.4$	380976	3589	1662	1627
$w_1=0.9; w_2=0.1$	265531	8700	2171	1644

Figure 8 shows four exemplary Pareto fronts for basic time cost B against the monetary cost M. Each example is done for a specific value of w_3 and is generated by increasing the weight for basic time cost w_1, while reducing the weight of the monetary cost w_2. The Pareto fronts show that both cost factors are competing and can be tuned with w_1 and w_2 for all values of w_3. Additionally, Table 5 shows the reduced waiting time with increasing w_3 for different combinations of w_1 and w_2 again for four examples. Without considering the utilization cost for all combinations of w_1 and w_2, the waiting time is much longer than the basic time T. Even for small values of w_3 the waiting time reduces fast and remains stable, demonstrating that the utilization penalty effectively reduces waiting time due to blocking in simulation.

V. CONCLUSION

In this work, we present a modeling framework for the optimal placement of workflows in serverless fog computing architectures. The framework jointly models workflow placement and selection within the architecture and formulates a nonlinear optimization problem that minimizes both monetary cost and execution time. To address decomposed information and the exponential growth of computational demand with increasing numbers of nodes, we introduce a decomposition strategy enabling scalable optimization. We demonstrate the advantages of the proposed method compared to a centralized formulation and heuristic baseline, as well as its adaptability through a Pareto analysis of competing cost objectives.

Future work will extend the framework toward probabilistic online selection strategies and incorporate additional cost factors relevant to practical deployment scenarios, such as workflow cold-start delays and energy consumption of serverless nodes.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE MANUSCRIPT PREPARATION PROCESS

During the preparation of this work the authors used GPT 5.0-5.3 in order to make adjustments to existing text in this paper, as well as small adaptation to coding sections for data storage and figure creation. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

[1] Saman Akbari and Manfred Hauswirth. *Hiku: Pull-Based Scheduling for Serverless Computing*. 2025.

[2] Muhamed Alarbi, Robert Belson, and Hanan Lutfiyya. ''ECO: edge continuum orchestrator framework for managing serverless chains across the cloud-edge spectrum''. In: *2024 33rd International Conference on Computer Communications and Networks (ICCCN)*. 2024.

[3] Daniel Barcelona-Pons et al. ''On the FaaS Track: Building Stateful Distributed Applications with Serverless Architectures''. In: *Proceedings of the 20th International Middleware Conference*. Davis CA USA, Dec. 2019, pp. 41–54.

[4] Janos Czentye et al. ''Optimizing Latency Sensitive Applications for Amazon's Public Cloud Platform''. In: *2019 IEEE Global Communications Conference (GLOBECOM)*. 2019, pp. 1–7.

[5] Simon Eismann et al. ''The State of Serverless Applications: Collection, Characterization, and Community Consensus''. In: *IEEE Transactions on Software Engineering* 48.10 (Oct. 2022), pp. 4152–4166.

[6] Christodoulos A. Floudas and Xiaoxia Lin. ''Mixed Integer Linear Programming in Process Scheduling: Modeling, Algorithms, and Applications''. In: *Annals of Operations Research* 139.1 (Oct. 2005), pp. 131–162.

[7] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2024. URL: https://www.gurobi.com.

[8] Josef Kallrath. ''Combined strategic and operational planning - an MILP success story in chemical industry''. In: *OR Spectrum* 24.3 (Aug. 2002), pp. 315–341.

[9] Jan Kronqvist et al. ''A review and comparison of solvers for convex MINLP''. In: *Optimization and Engineering* 20.2 (June 2019), pp. 397–455.

[10] LinsongGuo et al. *lzjzx1122/FaaSFlow: FaaSFlow v1.1*. Jan. 2022.

[11] Ashraf Mahgoub et al. ''{SONIC}: Application-aware data passing for chained serverless applications''. In: *2021 USeNIX annual technical conference*. 2021, pp. 285–301.

[12] R. Timothy Marler and Jasbir S. Arora. ''The weighted sum method for multi-objective optimization: new insights''. In: *Structural and Multidisciplinary Optimization* 41.6 (June 2010), pp. 853–862.

[13] P. Ngatchou, A. Zarei, and A. El-Sharkawi. ''Pareto Multi Objective Optimization''. In: *Proceedings of the 13th International Conference on, Intelligent Systems Application to Power Systems*. Arlington, Virginia, USA, 2005, pp. 84–91.

[14] Tobias Pfandzelter and David Bermbach. ''tinyFaaS: A Lightweight FaaS Platform for Edge Environments''. In: *2020 IEEE International Conference on Fog Computing (ICFC)*. Sydney, Australia, Apr. 2020, pp. 17–24.

[15] Arokia Paul Rajan. ''A review on serverless architectures - function as a service (FaaS) in cloud computing''. In: *TELKOMNIKA (Telecommunication Computing Electronics and Control)* 18.1 (Feb. 2020), p. 530.

[16] Sashko Ristov and Philipp Gritsch. ''FaaSt: Optimize makespan of serverless workflows in federated commercial FaaS''. In: *2022 IEEE International Conference on Cluster Computing (CLUSTER)*. 2022, pp. 183–194.

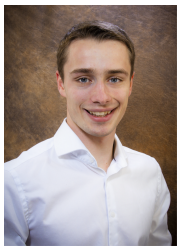
[17] Alireza Sahraei et al. ''XFaaS: Hyperscale and Low Cost Serverless Functions at Meta''. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz Germany, Oct. 2023, pp. 231–246.

[18] R. Schultz, L. Stougie, and M. H. Van Der Vlerk. ''Two-stage stochastic integer programming: a survey''. In: *Statistica Neerlandica* 50.3 (Nov. 1996), pp. 404–416.

[19] Márk Szalay, Péter Mátray, and László Toka. ''Real-Time FaaS: Towards a Latency Bounded Serverless Cloud''. In: *IEEE Transactions on Cloud Computing* 11.2 (Apr. 2023), pp. 1636–1650.

[20] Mohit Tawarmalani and Nikolaos V. Sahinidis. *Convexification and global optimization in continuous and mixed-integer nonlinear programming: theory, algorithms, software, and applications*. eng. Nonconvex

Optimization and its Applications Volume 65. Boston, [Massachusetts], 2002.
[21] Kan Wang, Jing Ma, and Edmund M-K Lai. ''BriskChain: Decentralized function composition for high-performance serverless computing''. In: *IEEE Access* 12 (2024), pp. 131313–131322.
[22] Wei Wei et al. ''Efficient stochastic scheduling for highly complex resource placement in edge clouds''. In: *Journal of Network and Computer Applications* 202 (June 2022), p. 103365.
[23] Renchao Xie et al. ''Workflow Scheduling in Serverless Edge Computing for the Industrial Internet of Things: A Learning Approach''. In: *IEEE Transactions on Industrial Informatics* 19.7 (July 2023), pp. 8242–8252.
[24] Murat Yilmaz et al. ''A Multivocal Literature Review of Function-as-a-Service (FaaS) Infrastructures and Implications for Software Developers''. In: *Systems, Software and Services Process Improvement*. 2020, pp. 58–75.



JOSHUA ADAMEK received the master's degree in electrical engineering from TU Dortmund, Germany, in 2022, where he is currently working toward his PhD degree with the Chair of Process Automation Systems. His research focuses on optimization for control of complex systems and optimal scheduling of serverless applications.

NATALIE CARL received the master's degree in information systems management from TU Berlin, Germany, in 2024, where she is currently working toward his PhD degree with the Scalable Software Systems research group. Her research focuses on optimizing serverless applications and platforms.



TREVER SCHIRMER received the master's degree in information systems management from TU Berlin, Germany, in 2022, where he is currently working toward his PhD degree with the Scalable Software Systems research group. His research focuses on optimizing serverless applications and platforms.



MORITZ HEINLEIN received the master's degree in biochemical engineering from TU Dortmund, Germany, in 2021, where he is currently working toward his PhD degree with the Chair of Process Automation Systems. His research is focused on tractable robust and stochastic formulations for model predictive control with a large uncertainty space.



DAVID BERMBACH earned a diploma degree in business engineering in 2010 and a PhD with distinction in computer science in 2014, both from Karlsruhe Institute of Technology, Germany. He then joined Technische Universität Berlin, Germany, first as a Postdoctoral Researcher, later as an Assistant Professor, and has now been heading the EECS department's Scalable Software Systems Research Group as a Full Professor since 2023. His research interests include the design and experiment-driven evaluation of large-scale software systems, currently particularly in the context of serverless computing. He serves in a variety of academic roles, including currently as an Associate Editor of the Journal of Parallel and Distributed Computing.



SERGIO LUCIA , Member IEEE, received the M.Sc. degree in electrical engineering from the University of Zaragoza, Spain, in 2010, and the Dr.-Ing. degree in optimization and automatic control from TU Dortmund University, Germany, in 2014. He joined Otto-von-Guericke Univer- sität Magdeburg and visited the Massachusetts Institute of Technology as a Postdoctoral Fellow. From 2017 to 2020, he was an Assistant Professor with TU Berlin. Since 2020, he has been a Professor with TU Dortmund University and the Head of the Chair of Process Automation Systems. His research interests include decision-making under uncertainty, distributed control, and the interplay between machine learning techniques and control theory. He is currently an Associate Editor of the Journal of Process Control.